ANITRACK

Web and Android Application Technical Documentation

Final Examination and Final Project

Bachelor of Science in Information Technology

BSIT 4-B | Group 2

Prepared by

Ian Mark Buenaflor

Kenji Agustine Caayohan

Rochelle Andales

Kent Jeced Alcantara

Group Leader Ian Mark Buenaflor

November 6, 2026

Submission Period November 16-20, 2026

ABSTRACT

AniTrack is a personal anime management system developed as a web application and an installable Android application. It allows a verified account holder to maintain a watchlist, record viewing progress, assign ratings, mark favorites, organize private collections, discover titles, and use an anime identification and recommendation assistant. The project extends the same application to Android through Capacitor while retaining a shared backend and database.

The implementation uses Vue and Quasar for the interface, Express and TypeScript for the application programming interface, and Prisma with PostgreSQL for persistent records. Kitsu supplies anime metadata, Gemini supports the assistant, and configurable SMTP supports account verification and password recovery. Authentication and verified-email requirements are enforced by both navigation guards and backend authorization. The Android application includes bundled frontend assets and uses native HTTP to communicate with the backend.

Verification records include 123 backend unit tests, 25 frontend unit tests, six isolated PostgreSQL integration cases, and 20 browser regression tests, together with a separate real browser and database journey. Physical-device checks were performed on an Infinix Note 30 running Android 14. A subsequent native outage check confirmed that account navigation remained responsive while the backend was stopped and that explicit reconnection succeeded after service restoration. These results support the reviewed local implementation. Actual Gmail inbox receipt and the required desktop Capacitor setup screenshot remain completion checks. The supplied APK is a local debug build that requires a reachable laptop backend; Internet hosting and production distribution are separate release activities.

Keywords: anime management, web application, Android, Capacitor, verification, software testing

CONTENTS

Abstract . . . . . **i**
Chapter 1 Introduction **1**
Background of the Project 1
Problem Statement 1
Objectives of the Project 1
Scope and Limitations 2
Chapter 2 Requirements and Project Scope **3**
Intended Users 3
Quality Requirements 3
Functional Requirements 3
Technology Selection 4
Chapter 3 System Architecture and Data Design **6**
Application Architecture 6
Core Data Relationships 7
Data Model Summary 7
Authentication and Verification 8
Authorization and Request Controls 9
Chapter 4 Application Features and User Guide **10**
Account Access 10
Overview and Spotlight 10
Watchlist and Viewing Progress 11
Private Collections 12
Discover and Anime Details 13
Anime Assistant 14
Calendar and Account Settings 14
Chapter 5 Android Implementation and Operation **15**
Capacitor Integration 15
Connection Failure and Recovery 15
Installed Application and Shared Features 16
Mobile Layout and Navigation 17
Forms and Account Recovery 18
Chapter 6 Testing and Evaluation **19**
Verification Procedure 19
Functional Acceptance Evidence 20
Interpretation of the Results 20
Chapter 7 Installation Operation and Maintenance **21**
Prerequisites 21
Starting the Local Website 21
Environment Configuration 21
Building and Installing Android 22
Network Operation and Maintenance 23
Chapter 8 Conclusions and Recommendations **24**
Conclusions 24
Recommendations and Remaining Work 24
References **25**
Appendix A Instructor Procedure and Submission Compliance **26**
Eight Step Capacitor Procedure 26
Assessment Evidence Mapping 27
Appendix B Build Provenance and Evidence Notes **28**
Reviewed Source and Package 28
Evidence Integrity and Remaining Verification 28

LIST OF FIGURES

Figure 1 System architecture 6
Figure 2 Core library relationships 7
Figure 3 Required account verification flow 8
Figure 4 Web overview and saved library summary 10
Figure 5 Web watchlist with status and library controls 11
Figure 6 Web collection with fanned cover preview 12
Figure 7 Anime detail page with an accessible trailer action 13
Figure 8 Installed Android overview and watchlist 16
Figure 9 Android Collections and revised landing page 17
Figure 10 Android login and password recovery forms 18

LIST OF TABLES

Table 1 Principal functional requirements 4
Table 2 Reviewed technology stack 5
Table 3 Persistent data models 8
Table 4 Recorded verification results 19
Table 5 Selected behavior and acceptance checks 20
Table 6 Principal environment settings 22
Table 7 Compliance with the instructor procedure 26
Table 8 Required submission evidence 27
Table 9 Source and build identification 28

CHAPTER 1
INTRODUCTION

Background of the Project

Anime viewers may follow several series at the same time, pause a title, or save recommendations for later. A personal tracking application can organize these decisions around a consistent record of what the viewer has watched and what the viewer intends to watch next. AniTrack addresses this use case through an account-based library rather than through a public streaming service.

The project has two required outputs: a working web application and a functional Android application created from the existing project using Capacitor. The course instructions accept a locally hosted website and require evidence that the mobile application is installed on a physical phone and that backend-dependent features connect correctly. This documentation therefore explains both the application design and the evidence supporting the submitted local build (Final Examination and Final Project Instructions, 2026).

AniTrack uses one set of frontend screens across the browser and Android runtime. The web client runs as a single-page application, while the Android package contains compiled frontend assets. Both clients communicate with the same API. Changes to a watchlist or collection are stored in the shared database and become available to the other client after a fresh read. The Android package does not contain the backend or database.

Problem Statement

The project is concerned with maintaining a coherent personal anime record across web and mobile access. The system must support progress updates and organization without exposing private account data to other users. It must also distinguish public product information from application functions that require a verified account.

The main engineering questions are whether a user can reliably manage the same library through both clients, whether authentication and ownership checks protect private resources, and whether the Android application remains understandable when its backend is unavailable. These questions guided the implemented modules and the recorded verification work.

Objectives of the Project

The general objective is to provide an authenticated anime tracking system that operates through a web browser and an installable Android application using a shared backend.

The specific objectives are to provide registration with required email verification; maintain watchlist status, ratings, favorites, and episode progress; organize saved titles into private collections; support catalog discovery and anime details; assist identification and recommendation through conversational clues; display available episode air dates; provide account recovery and session controls; and verify the implementation through automated tests and physical-device checks.

Scope and Limitations

The reviewed scope includes the public informational landing page, account workflows, overview, watchlist, Collections, Discover, anime details, Anime Assistant, Calendar, and account settings. The Android scope includes bundled assets, the native networking path, responsive screens, application branding, session persistence, and a debug APK installed on a physical phone.

Discover, Anime Assistant, anime details, and the remaining workspace modules require authentication and verified email. The landing page describes these capabilities and links to account access. It does not expose usable catalog or assistant tools to guests.

The system is not an anime streaming platform. Trailers are shown through an available external video source, and episode information depends on provider coverage. Unknown air dates are displayed as unavailable rather than inferred. Assistant suggestions depend on the configured provider and may require additional clues. The current release does not claim offline library synchronization, push notifications, an iOS build, public deployment, or production signing.

CHAPTER 2
REQUIREMENTS AND PROJECT SCOPE

Intended Users

The primary application user is a person who wants to maintain a private anime library. A registered but unverified account may complete verification and permitted recovery operations. A verified member may access the personal workspace. The backend also contains administrative authorization controls, but this report does not present a separate administrative interface as a completed member-facing feature.

The development and assessment audience includes Group 2, other group members who must run or review the project, and the instructor evaluating the web and Android outputs. The original screenshots and installation files accompany this report so that an evaluator can inspect evidence beyond the printed figures.

Quality Requirements

Private operations must validate the account and the ownership of the requested resource on the server. A client-side redirect alone is insufficient to protect an API endpoint. Passwords must be stored as hashes, recovery tokens must be bounded and single use, and account changes must not allow an old session to repopulate another user's interface.

The interface must remain usable at narrow widths and with the physical phone's configured text scaling. Forms require visible labels, useful validation, accessible controls, and recoverable error states. A failed background session request must not prevent a guest from opening account or informational pages.

The submitted implementation must also be reproducible. Dependency lockfiles, an identifiable Android project, a recorded build variant, and a checksum for the packaged APK support this requirement. Verification records must distinguish automated provider fixtures, live integration checks, and actual inbox delivery.

Functional Requirements

Table 1 summarizes the principal functional requirements. The detailed procedures in Chapters 4 and 5 explain how these requirements are exposed to the user. The requirements describe the reviewed implementation and its intended access rules (Group 2, 2026a, 2026b).

Table 1 Principal functional requirements

ID	Requirement	Expected user outcome
FR1	Registration and verification	Create an account and confirm its email before entering the workspace.
FR2	Authentication and recovery	Sign in, restore a valid session, reset or change a password, and sign out.
FR3	Personal watchlist	Save or create a title and maintain status, rating, favorite state, and progress.
FR4	Private collections	Create groups, manage memberships and ordering, and delete a group without removing watchlist progress.
FR5	Discovery and details	Search and filter catalog titles, inspect available information, and import a title.
FR6	Anime Assistant	Submit remembered scenes or preferences, refine suggestions, and save a matching catalog title.
FR7	Calendar and profile	View available dates for Watching titles and maintain account information.
FR8	Functional Android client	Use the shared features from an installed APK connected to the backend.

Technology Selection

Vue provides a component-based interface model, while Quasar supplies the application framework and Capacitor build integration. Express supplies the HTTP routing and middleware layer. Prisma connects the application's model to database queries and migrations. These roles are consistent with the respective official documentation (Express.js Contributors, n.d.; Prisma, n.d.; Quasar Framework, n.d.; Vue.js, n.d.).

Table 2 Reviewed technology stack

Layer	Technology	Recorded version	Role in AniTrack
Interface	Vue	3.5.43	Component rendering and reactive interface state.
Application framework	Quasar and app-vite	2.35.2 and 3.10.3	Shared application shell and web or native asset builds.
Client state and routing	Pinia and Vue Router	3.0.4 and 5.4.0	Account and library state, navigation, and access guards.
Web bundler	Vite	8.3.5	Development assets and production compilation.
API	Express	5.3.0	HTTP endpoints, validation integration, and middleware.
Database access	Prisma Client	6.19.3	Typed queries, schema management, and migrations.
Persistent storage	PostgreSQL on Neon	Server version not recorded	Relational application records shared by both clients.
Account security	bcrypt and jsonwebtoken	6.0.0 and 9.0.3	Password hashing and access-token processing.
Email	Nodemailer	10.3.1	Configurable SMTP account messages.
Android runtime	Capacitor core and Android	8.5.3	Bundled web interface and native integration.

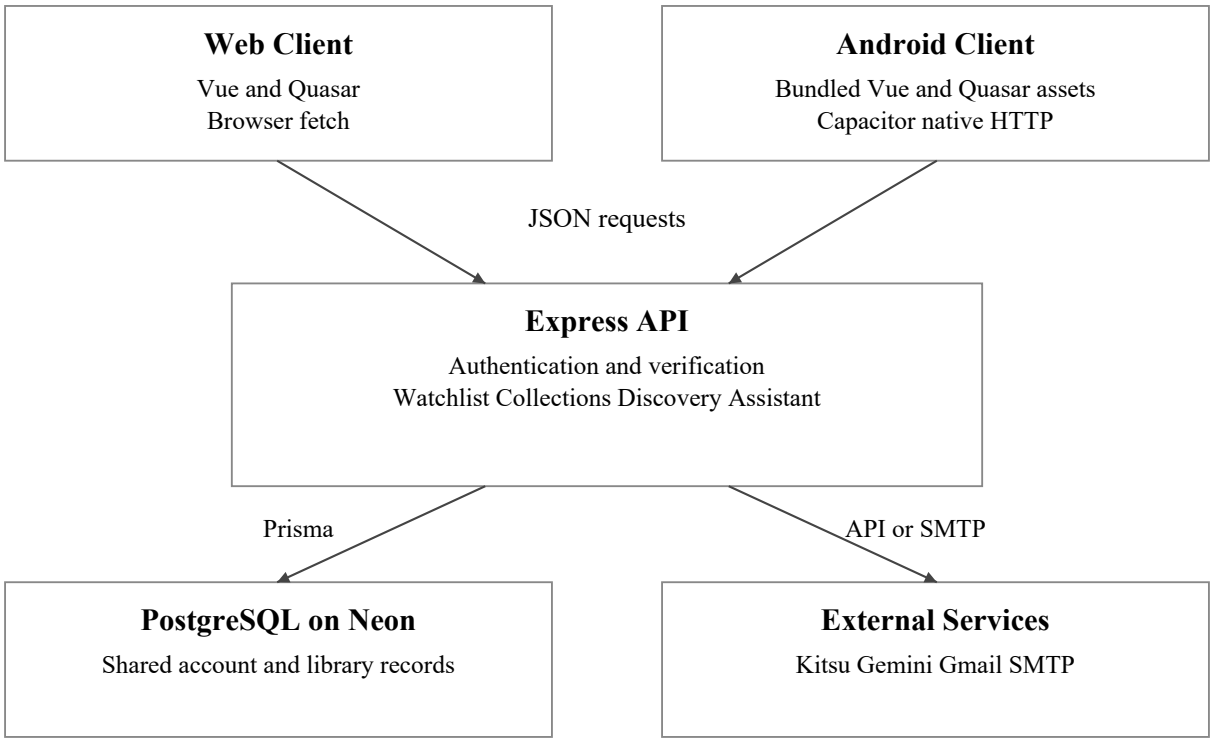
Source: Project lockfiles and the packaged build manifest, reviewed November 6, 2026. Versions describe this project, not a recommendation to upgrade dependencies.

CHAPTER 3
SYSTEM ARCHITECTURE AND DATA DESIGN

Application Architecture

Figure 1 presents the implemented logical architecture. The web and Android clients use the same backend functions but different transport paths. Browser requests use fetch through the configured API route. The Android client uses CapacitorHttp, which exposes native HTTP request helpers through the Capacitor runtime (Ionic, n.d.).

Figure 1 System architecture



Source: AniTrack client configuration, API routes, and native transport implementation.

The Express API performs authorization and invokes the relevant application service. Persistent library and account data are stored in PostgreSQL through Prisma. Provider requests are made for catalog information, assistant responses, and account email delivery. PostgreSQL supplies relational constraints and transaction support, while Prisma defines the application's models and migration workflow (PostgreSQL Global Development Group, n.d.; Prisma, n.d.).

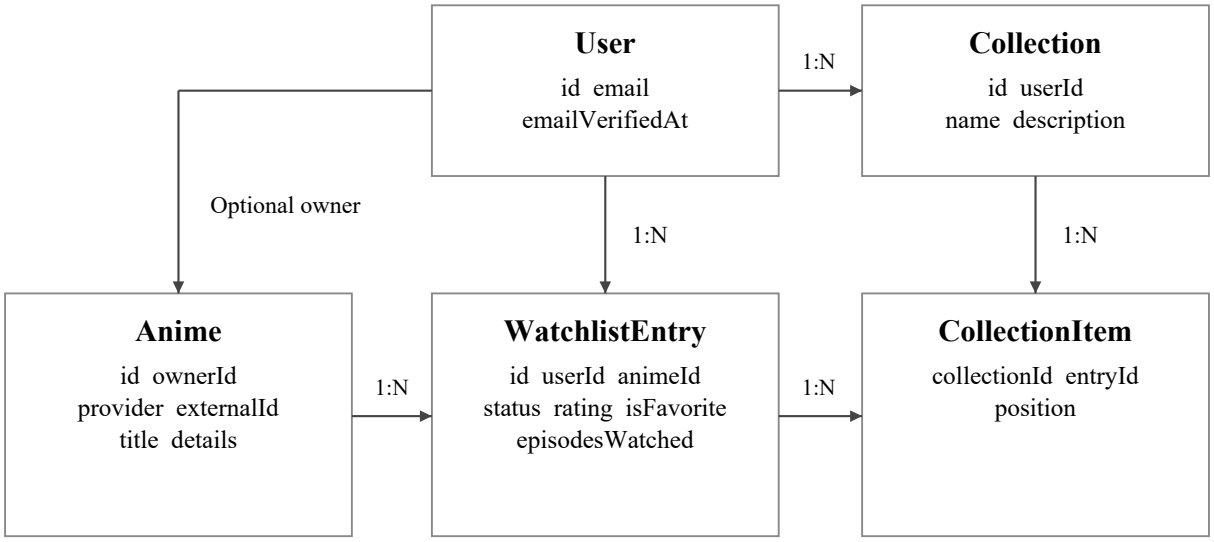
The tested local topology places the API on the laptop at port 3001. The web application uses port 5173, and the supplied APK addresses the laptop's configured Wi-Fi address. Closing the laptop, stopping the API, or changing to a network where that address is

unreachable prevents backend-dependent operations. This network dependency is separate from whether the frontend has been packaged inside the APK.

Core Data Relationships

Figure 2 shows the relationships that support the personal library. A user owns watchlist entries and collections. A watchlist entry links one user to one anime record. CollectionItem links a collection to a saved watchlist entry and records its position. A title can therefore participate in different collections while keeping one progress record for its owner.

Figure 2 Core library relationships



Authentication and usage models are summarized in Table 3.

Source: ANITRACK-BACKEND/prisma/schema.prisma. The 1:N labels indicate one-to-many relationships; the Anime owner is optional for catalog records.

The unique user-and-anime pair prevents duplicate entries in the same user's watchlist. A unique provider-and-external-ID pair identifies imported catalog records. The compound collection-and-entry key prevents duplicate memberships. Removing a collection removes its memberships, while the associated watchlist entry and progress remain available.

Data Model Summary

The remaining records support authentication, account recovery, and assistant usage control. Table 3 identifies all eight models in the reviewed Prisma schema. This is a model summary rather than a complete listing of every timestamp or generated field.

Table 3 Persistent data models

Model	Important keys or fields	Purpose
User	id; unique email; passwordHash; emailVerifiedAt	Account identity, verified status, role, and spotlight preference.
Anime	id; optional ownerId; provider and externalId	Imported metadata or a user-owned custom anime record.
WatchlistEntry	id; userId and animeId; status; rating; episodesWatched	The owner's saved state, favorite flag, and viewing progress.
Collection	id; userId; name; description	A private named grouping of saved anime.
CollectionItem	collectionId and entryId; position	Membership and ordering within a collection.
RefreshSession	id; tokenHash; familyId; userId; expiry and revocation	Refresh-token rotation and server-side session validity.
AccountToken	tokenHash; purpose; userId; expiresAt	Single-use email-verification or password-reset token.
AssistantUsageDay	day; attempts; reservedMicros; token totals	Shared daily assistant usage and budget accounting.

Authentication and Verification

Registration creates the user and a restricted session. A pending account is directed to the email-check workflow and cannot open the private workspace. A verification link must consume a valid token before emailVerifiedAt is populated. The API reads session and account state when authorizing protected requests, so an unverified account cannot obtain access merely by entering a private URL.

Figure 3 Required account verification flow



Source: Account service, authentication middleware, and frontend route guards.

The implementation uses bcrypt with a cost factor of 12 for passwords. Access tokens expire after 15 minutes, and refresh-session records support a 30-day lifetime with rotation and revocation. Stored refresh and account tokens use hashes. Verification tokens expire after 24 hours; password-reset tokens expire after 30 minutes. A verification resend cooldown

limits repeated requests. These values describe the reviewed source configuration (Group 2, 2026b).

Password recovery uses a web form reached through the emailed link. Resetting or changing the password revokes prior sessions as implemented by the account service. On Android, a user may complete verification or reset in the browser and return to the app; the email-check action then refreshes verification status. Automatic native deep linking is not required for this workflow.

Authorization and Request Controls

The backend applies authentication and verified-email checks independently of the menu. Resource operations use the authenticated user's identity to restrict private watchlists, custom titles, collections, and memberships. Origin checks, request-size limits, security headers, input validation, and rate limits add controls around those operations.

The security assessment supports the controls and cases that were tested. A dependency audit reporting no known advisories does not establish that the application has no vulnerabilities. The recorded ownership, token, recovery, and session tests provide more specific evidence than a general claim of complete security.

CHAPTER 4
APPLICATION FEATURES AND USER GUIDE

Account Access

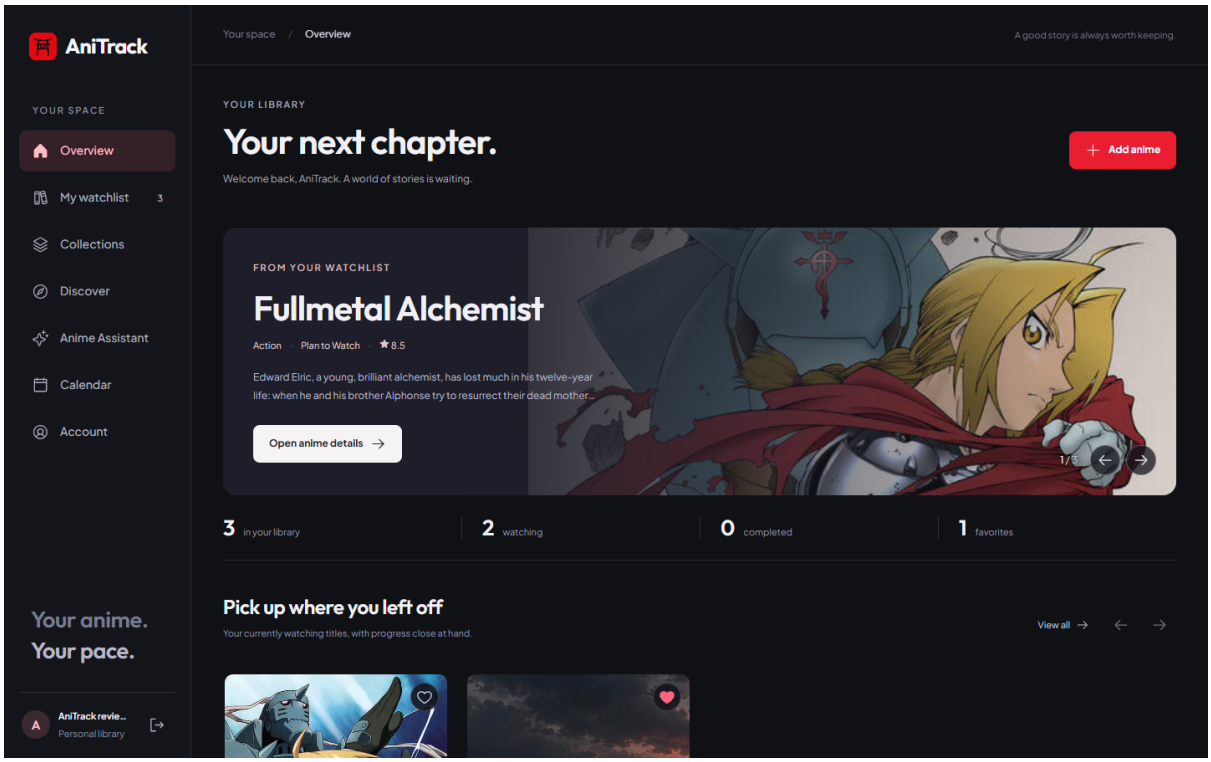
Create an account using a name, email address, and a password that meets the displayed requirements. Confirm the received verification link and return to check the updated status before entering the workspace. Returning members use the login form or a valid restored session. The recovery link requests a password-reset message. The personal Gmail receipt check remains separate from the captured-mail tests.

Overview and Spotlight

The overview summarizes the member's saved library and provides entry points to saved titles. Counts reflect the stored library, including relevant status and favorite totals. The spotlight displays a selected saved anime. New accounts receive an appropriate first-use state rather than fabricated library statistics.

To use the overview, sign in with a verified account, open a saved title or the add action, and continue to the relevant feature. A user may choose a spotlight title from the watchlist. Figure 4 shows the desktop overview using disposable verification data.

Figure 4 Web overview and saved library summary



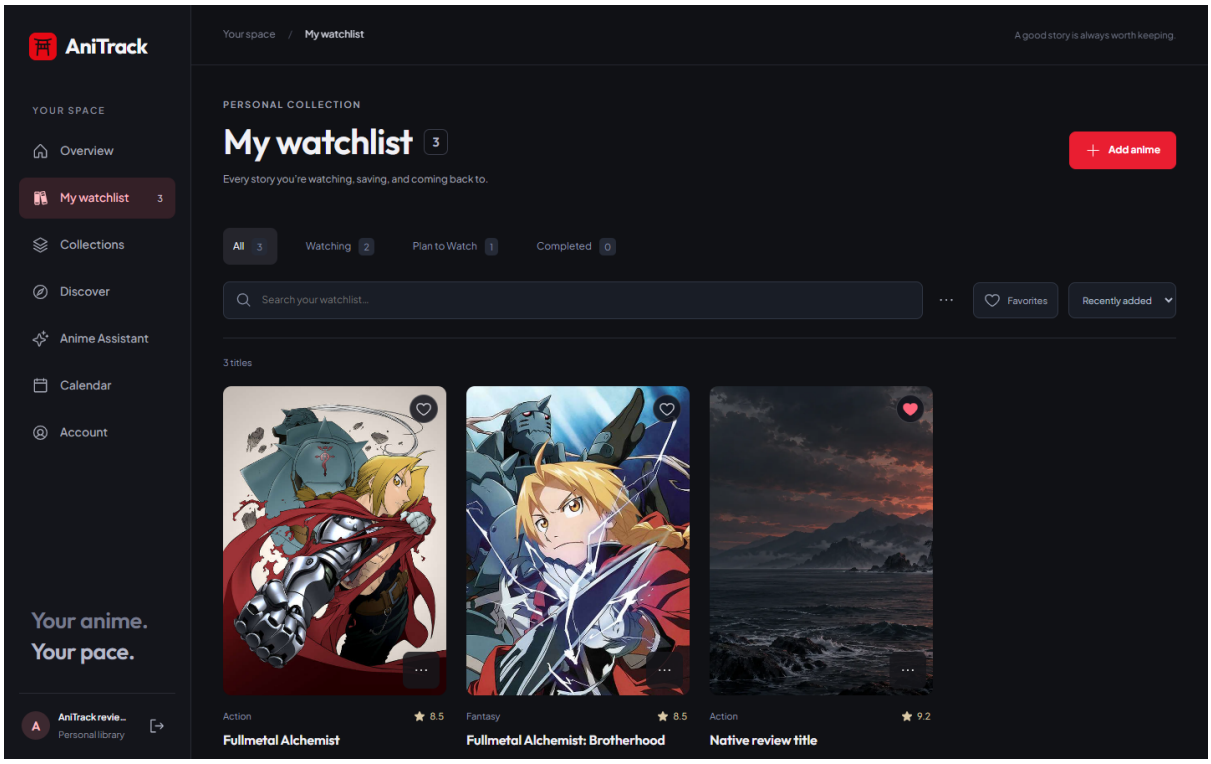
Source: Web application evidence captured November 5, 2026. The disposable account and test data were removed after verification.

Watchlist and Viewing Progress

The watchlist is the primary record of the member's anime activity. A title can be imported from the catalog or created as a custom entry. Available states include Watching, Completed, and Plan to Watch. The member can update watched episodes, assign a personal rating, toggle a favorite, search the saved library, apply available filters, and change the sort order.

After adding a title, select its appropriate status and record viewing progress. Use the episode controls or exact-count editor when a known episode count is available. Save or complete the edit and confirm the state after reloading. Removing a watchlist entry removes that personal saved record. The default reviewed limit is 500 saved titles, subject to backend configuration.

Figure 5 Web watchlist with status and library controls



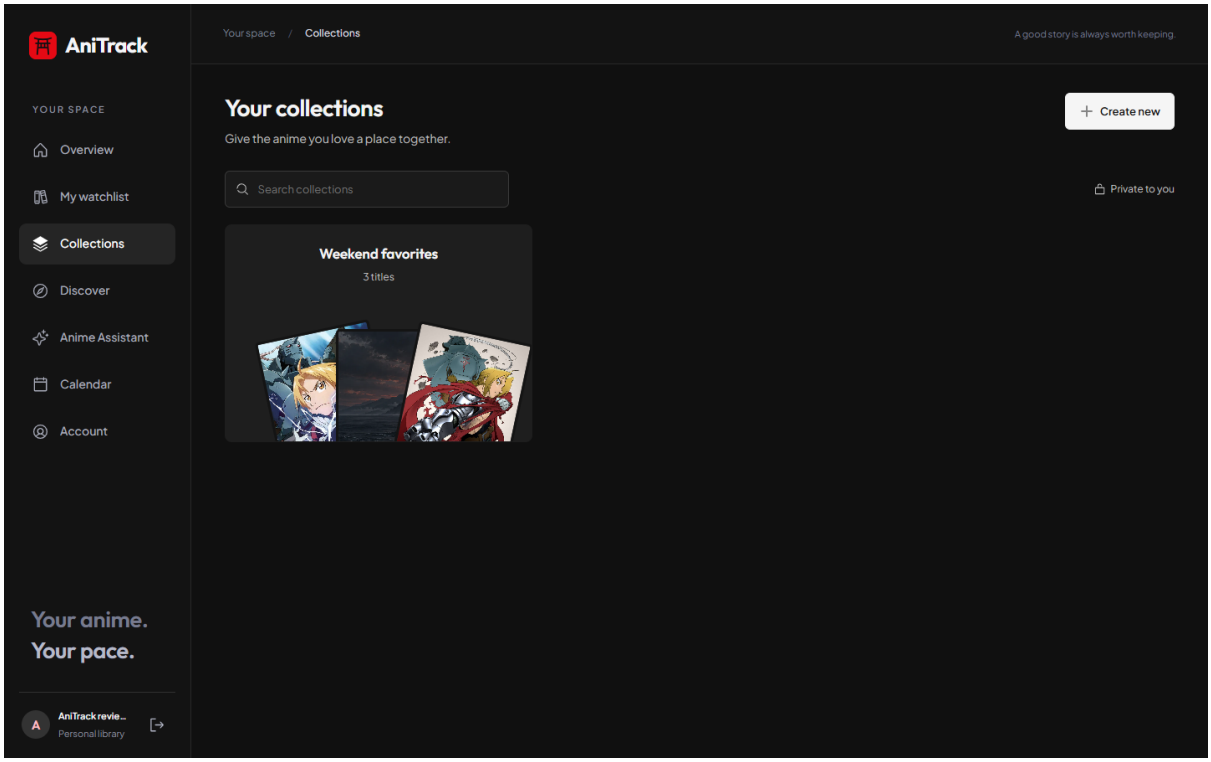
Source: Web application evidence captured November 5, 2026.

Private Collections

Collections organize existing watchlist entries into themes chosen by the member. A member can create a collection, supply a name and description, select saved anime, change the order, remove memberships, and delete the collection. The cover fan previews its contents using the established charcoal and neutral visual treatment.

To create a group, open Collections, select Create new, enter the collection information, and add entries from the personal watchlist. Reopen the group to rename it or adjust its contents. Deleting the group preserves the underlying watchlist and progress. The reviewed limits are 20 collections per account and 100 titles per collection.

Figure 6 Web collection with fanned cover preview



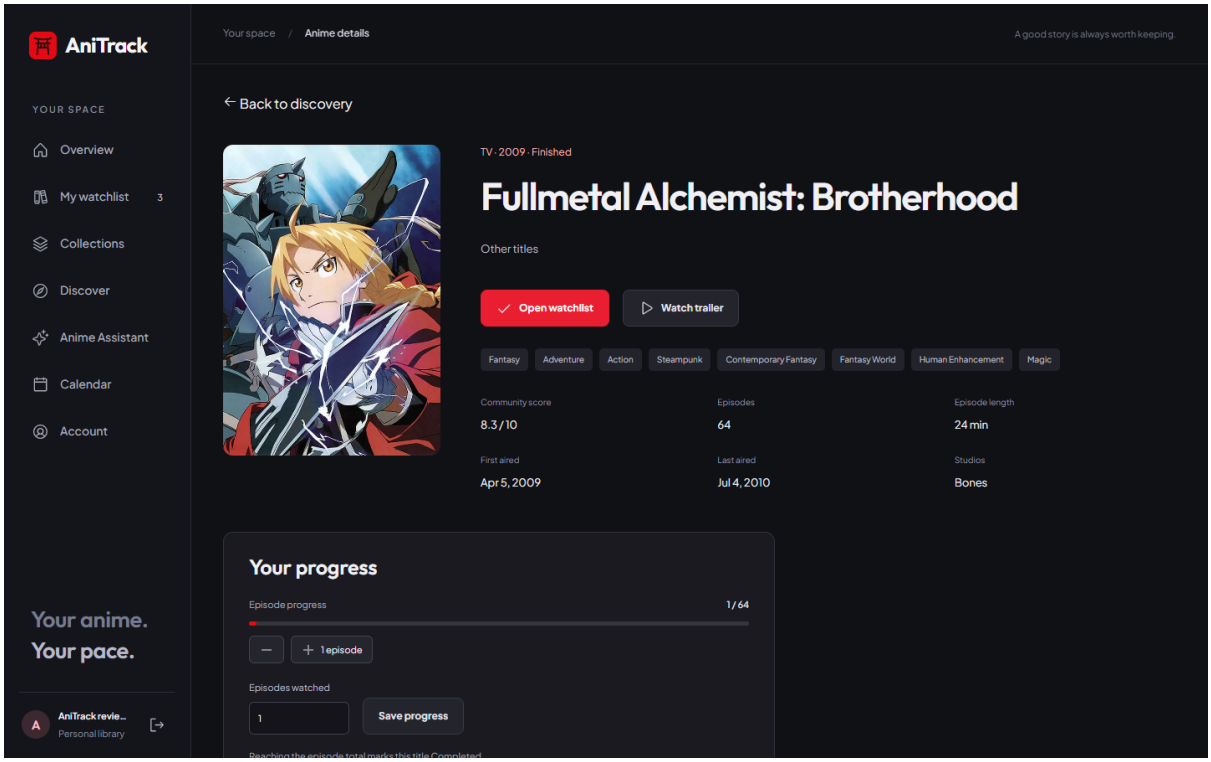
Source: Web application evidence captured November 5, 2026.

Discover and Anime Details

Discover helps a verified member find catalog titles by name, genre, or available browsing category, including popular, top-rated, seasonal, and upcoming views. Kitsu supplies the metadata used by this feature. Provider coverage and availability affect what information can be displayed.

Opening a title presents its available synopsis, facts, episodes, related entries, and similar titles. The trailer action appears with the main title actions before the long synopsis. The player opens in a responsive dialog with a close control and an external viewing fallback. A member can save the title to the watchlist and continue tracking it there.

Figure 7 Anime detail page with an accessible trailer action



Source: Web application evidence captured November 5, 2026. The original PNG accompanies the report for full-resolution review.

Anime Assistant

The assistant supports two related tasks: identifying an anime from remembered scenes and suggesting titles from preferences. Enter the available clues, submit the request, and refine the answer through follow-up details when needed. Suggestions resolve to catalog titles that can be inspected and saved.

The interface can hide already saved suggestions. Provider failures and configured usage limits are presented as recoverable conditions. Conversations remain in application memory and clear on reload or an account change. The feature is not a persistent social chat or a guarantee that an incomplete description will identify one exact title.

Calendar and Account Settings

Calendar displays available episode dates associated with Watching titles. If a provider has not announced or supplied a date, the application explains the absence rather than inventing a schedule. This module does not send release notifications.

Account settings allow a member to maintain a name, review verification status, change a password, and request permanent deletion with explicit confirmation. Deletion is an account operation with consequences for that user's records. Verification data used in testing was removed separately from the user's real account.

CHAPTER 5

ANDROID IMPLEMENTATION AND OPERATION

Capacitor Integration

Capacitor provides the native runtime used to package the existing web interface for Android. Quasar's Capacitor mode builds the frontend assets and places them inside the native project. This approach preserves the shared application while supplying an Android package and native APIs (Ionic, n.d.; Quasar Framework, n.d.).

The project uses the identifier org.anitrack.app and the display name AniTrack. The native project resides in src-capacitor/android. The supplied debug APK contains compiled assets and uses the same red torii mark as the website. Asset preparation alone is not treated as evidence of an installable APK; the native Gradle build and phone installation were also completed.

Connection Failure and Recovery

The supplied APK targets a local laptop API. It can display bundled pages while that API is unavailable, but operations that require account or library data still need the backend. The application explains this local-build requirement and provides an explicit retry action.

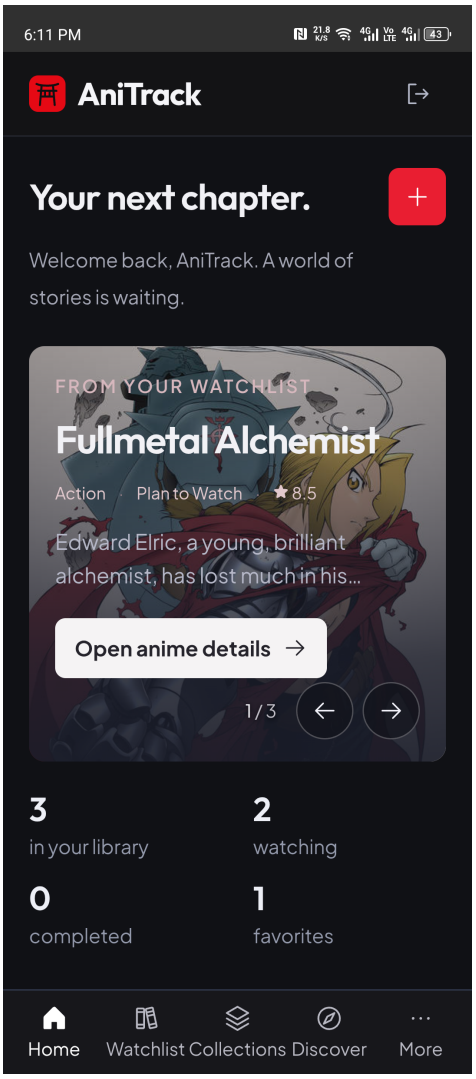
Public and account navigation no longer waits for a background session request. Session refresh has an eight-second deadline, and the native adapter respects cancellation even if the platform request has not returned. A failed restore preserves existing account state rather than treating every network failure as an explicit sign-out.

In the November 6 physical-device check, the API was briefly stopped. Registration opened in approximately 1.15 seconds, and registration, login, and recovery navigation together took approximately 1.63 seconds. After the API was restored, the retry completed in approximately 0.25 seconds. These are observed timings from that check, not a general performance benchmark (Group 2, 2026c).

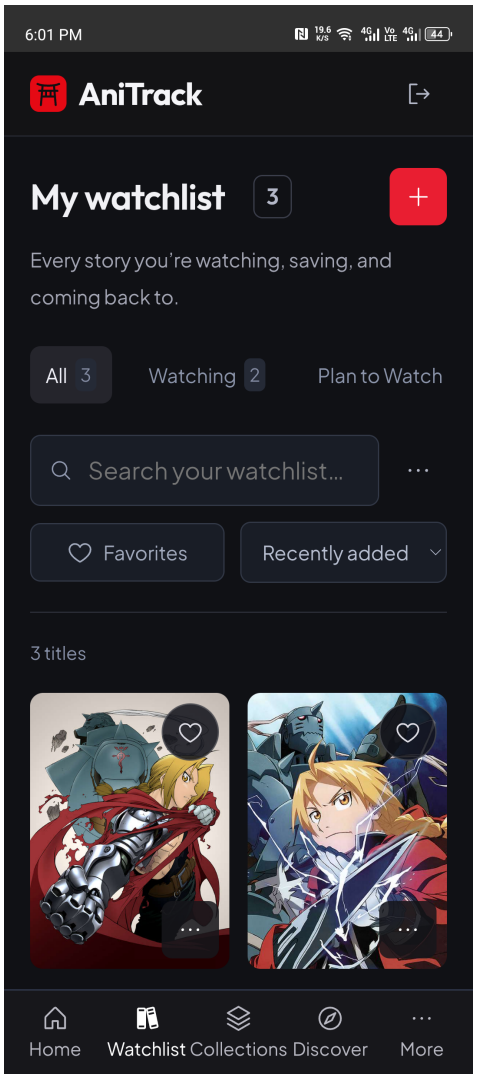
Installed Application and Shared Features

The tested physical device is an Infinix Note 30, model X6833B, running Android 14. The installed application uses the shared backend and database. Watchlist edits, collection operations, and account changes therefore use the same stored records as the website, subject to authorization and a reachable API.

Figure 8 Installed Android overview and watchlist



Android overview



Android watchlist

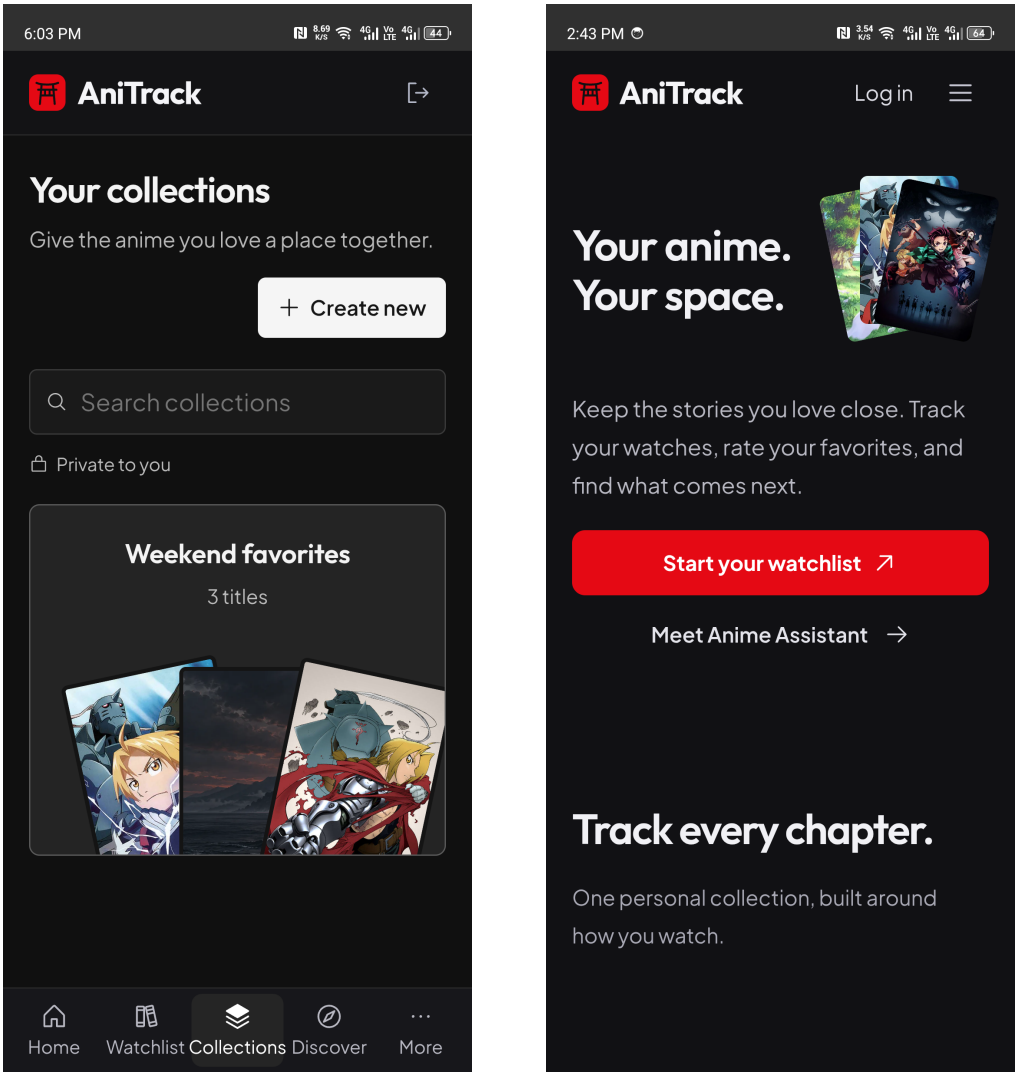
Source: Physical-phone captures from November 5, 2026. These images show the installed application, not browser device emulation.

Mobile Layout and Navigation

The phone layout uses a compact application shell, touch controls, scrolling dialogs, and space for native screen insets. The private workspace uses bottom navigation for common destinations and a More menu for additional modules. The public header uses one row with a compact menu. Its hero places a small fan of covers beside the heading, leaving the copy and actions on a plain background.

The retained phone text scale is 1.15. During the follow-up check, the Android WebView measured 360 CSS pixels in width without horizontal overflow. The two-line mobile heading and the primary action remained visible. Figure 9 pairs the private Collections view with the revised public landing page.

Figure 9 Android Collections and revised landing page



Android Collections

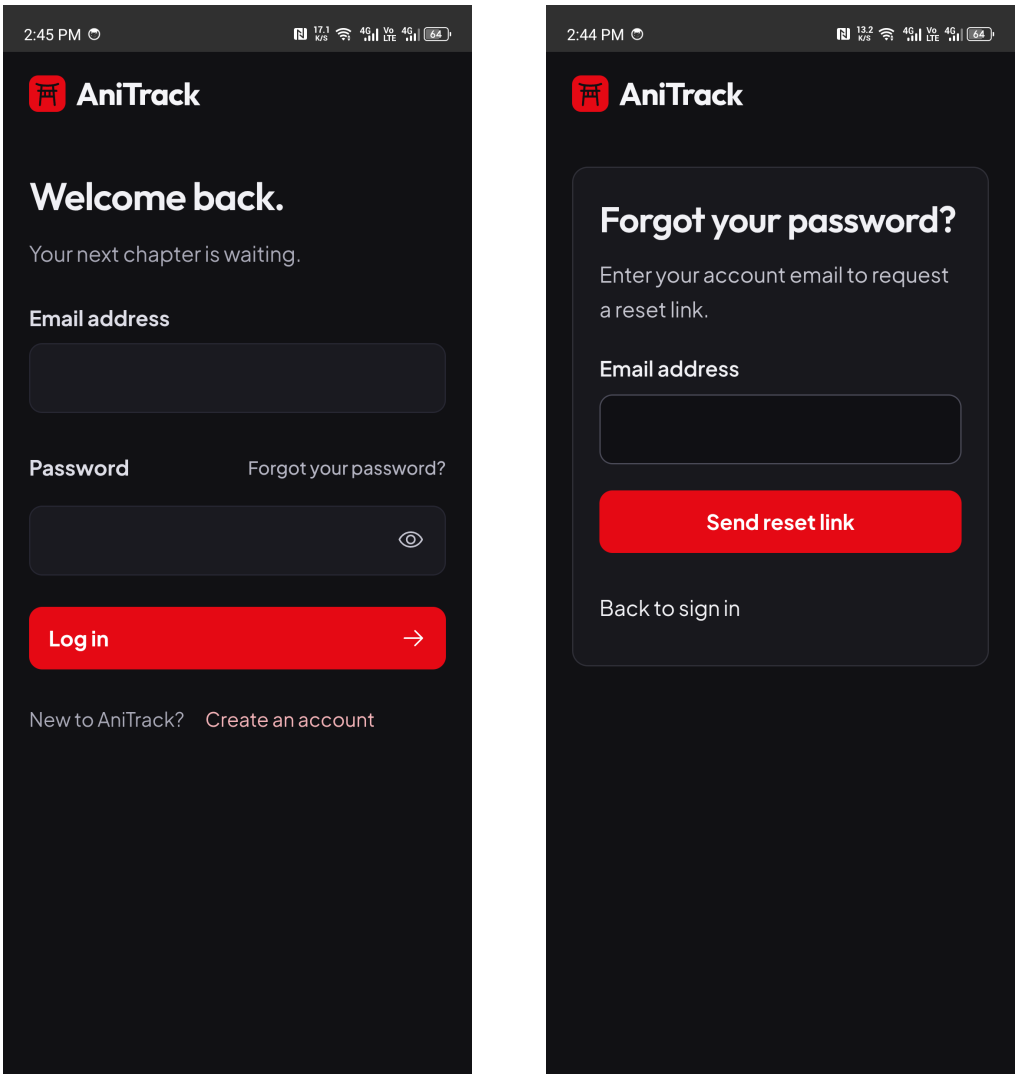
Android landing page

Source: Physical-phone evidence. Collections was captured November 5 and the revised landing was captured November 6, 2026.

Forms and Account Recovery

The account screens prioritize the form and remove the repeated landing navigation. The password-recovery link is associated with the password field in the login form. Visible labels and password-visibility controls remain available, while input sizing supports the phone's text scale.

Figure 10 Android login and password recovery forms



Android login Android recovery

Source: Physical-phone captures from November 6, 2026. No password, verification token, or recovery token is visible.

CHAPTER 6
TESTING AND EVALUATION

Verification Procedure

Verification combined unit tests, isolated database integration tests, browser regressions, a real browser and database account journey, production layout inspection, and physical Android checks. Deterministic tests used controlled provider responses where appropriate. Separate bounded live checks exercised Kitsu metadata and Gemini suggestions.

Disposable accounts and uniquely owned database schemas limited the effect of testing on normal application records. The real account journeys removed their test users and schemas after completion. Captured messages established token and link behavior without sending mail to fabricated addresses. The required screenshots were retained as assessment evidence, while temporary review folders, traces, and helper files were removed.

Table 4 Recorded verification results

Verification layer	Recorded result	Evidence date
Backend unit tests	123 tests passed; lint and TypeScript build passed.	November 5, 2026
Frontend unit tests	25 tests passed; lint, typecheck, and SPA build passed.	November 6, 2026
PostgreSQL integration	Six isolated cases passed; owned test schemas removed.	November 5, 2026
Compiled-site browser suite	20 regressions passed; one optional live test skipped in this suite.	November 6, 2026
Separate real browser journey	Account, captured verification, library, Collections, profile, and deletion journey passed.	November 5, 2026
Android build and installation	Gradle assembleDebug succeeded and the debug APK was installed on the physical phone.	November 6, 2026
Native outage check	Account navigation stayed responsive and explicit retry recovered after API restoration.	November 6, 2026
Production visual inspection	Zero failures and three reviewed warnings at desktop, tablet, and phone widths.	November 6, 2026

Source: Execution audits, validation record, and APK build manifest. Counts from different test layers are not combined into a single coverage percentage.

Functional Acceptance Evidence

The principal findings concern the expected behavior of the reviewed application. Table 5 connects selected user requirements to the recorded checks. The evidence includes both successful flows and negative access cases; an interface being visible is not treated as proof that its protected API is authorized.

Table 5 Selected behavior and acceptance checks

Area	Expected behavior	Recorded outcome
Private access	Guests and pending accounts cannot open workspace functions.	Browser and backend cases passed.
Verification and recovery	Valid tokens authorize the intended action and cannot be reused as unrestricted credentials.	Captured-mail and isolated account cases passed.
Ownership	Another user's identifiers do not authorize access to private resources.	Isolated authorization cases passed.
Shared library	Status, favorite, rating, progress, and collection changes persist between clients.	Real browser and native checks passed.
Discovery and assistant	Known provider data can be inspected and a suggested title saved.	Bounded live provider journeys passed.
Trailer and mobile controls	Main actions, dialogs, forms, and navigation remain accessible.	Browser regressions and recorded phone checks passed.
Backend outage	Public navigation remains responsive and retry recovers when service returns.	Stalled-request regression and real native outage check passed.
Personal Gmail receipt	Verification and reset messages arrive and their received links work.	Awaiting the user's real-inbox check.

Interpretation of the Results

The tests support the functional local web and Android implementation at the recorded revisions. The final source review retained the account-verification requirement and private application boundary. The November 6 regression work also addressed the observed freeze during unavailable-server navigation and the decorative artwork layer that intercepted a reconnect control.

Visual checks reported three warnings concerning the existing desktop typography scale, the absence of video footage, and restrained motion. These were documented as retained design decisions. The phone layout was checked with the device's configured text scaling rather than reducing that user preference to make the layout fit.

CHAPTER 7

INSTALLATION OPERATION AND MAINTENANCE

Prerequisites

The development environment requires Node.js and npm, the project-local Quasar CLI, Java JDK 21, Android Studio, the Android SDK, and platform-tools for ADB. The reviewed Android build uses Gradle 8.14.3, compile and target SDK 36, and minimum SDK 24. The tested physical device runs Android 14; the minimum SDK declaration alone is not a test result for every older device.

The frontend and backend dependency trees are installed from their lockfiles. Native packages have their own lockfile under src-capacitor. Backend secrets remain in the private backend environment file. The frontend receives only public connection configuration and must not contain SMTP credentials, database passwords, or the Gemini API key.

Starting the Local Website

Open the backend folder, retain its configured private environment, and install or restore the project dependencies. Generate the Prisma client and deploy the existing migrations. The commands below do not reset or seed the existing database.

```
cd ANITRACK-BACKEND
npm ci
npm run db:generate
npm run db:deploy
npm run dev
```

Open a second terminal in the frontend folder and run the application. The frontend folder contains package.json and quasar.config.ts.

```
cd ANITRACK-FRONTEND
npm ci
npm run dev
```

Use http://localhost:5173 on the laptop. The health endpoint at http://localhost:3001/api/health verifies that the API can query its database. Keep both servers running during normal local use.

Environment Configuration

Table 6 identifies the connection and service settings needed for operation. It intentionally describes variable names and purposes without reproducing private values. SMTP is configured through Nodemailer; its transport supports an immediately secured connection on port 465 and a configurable sender account (Nodemailer, n.d.).

Table 6 Principal environment settings

Setting	Location	Purpose
DATABASE_URL and DIRECT_URL	Backend	Database access and migration connection configuration.
JWT_SECRET	Backend	Secret used to sign and validate access tokens.
CLIENT_ORIGIN	Backend	Trusted browser origin for the web application.
ACCOUNT_LINK_ORIGIN	Backend	Optional exact origin for verification and reset pages reachable from the phone.
SMTP_HOST SMTP_PORT SMTP_SECURE	Backend	Mail server address, port, and TLS mode.
SMTP_USER SMTP_PASSWORD MAIL_FROM	Backend	Private email credentials and configured sender address.
GEMINI_API_KEY and model settings	Backend	Assistant provider access and configured model behavior.
VITE_API_URL	Frontend	Public web API base or the existing same-site API route.
ANITRACK_DEV_HOST	Frontend	Development host binding; 0.0.0.0 permits the trusted Wi-Fi interface.
VITE_MOBILE_API_URL	Frontend build	Backend API address embedded in the native build.

The tested Gmail connection used port 465 with TLS after port 587 timed out on the available network. App-password display spaces were removed from the private configuration. This describes a successful authentication check; it does not replace the required verification and password-reset inbox test.

Building and Installing Android

Capacitor mode is already installed. Restore its existing dependencies rather than adding the mode repeatedly. The core, CLI, and Android packages should retain compatible major versions. Appendix A records the instructor's eight steps and the actual verification outcome for each.

```
npm ci --prefix src-capacitor
npm list --prefix src-capacitor @capacitor/core
npm list --prefix src-capacitor @capacitor/cli @capacitor/android
```

Stop the regular frontend asset server before compiling shared Quasar output. Set the JDK and SDK for the current shell and supply the laptop address for a local debug build. The placeholder below must be replaced with the laptop's current Wi-Fi address.

```
$env:JAVA_HOME='C:\Program Files\Microsoft\jdk-21.0.13.21-hotspot'
$env:ANDROID_HOME="$env:LOCALAPPDATA\Android\Sdk"
$env:VITE_MOBILE_API_URL='http://YOUR_LAPTOP_IP:3001/api'
npm run prepare:android
cd src-capacitor/android
.\gradlew.bat --no-daemon --max-workers=1 `
  '-Dorg.gradle.jvmargs=-Xmx1024m -XX:MaxMetaspaceSize=384m' `
  assembleDebug
```

The generated file is app/build/outputs/apk/debug/app-debug.apk relative to the Android project. The supplied copy is apk/AniTrack-1.0-local-debug.apk. Installation used the paired physical phone over wireless ADB because a USB cable was unavailable. An explicit device selector identifies that connection; the instructor's adb -d form specifically selects a USB device.

Network Operation and Maintenance

The packaged local API address is http://192.168.254.142:3001/api. For phone-browser account links, the local website address is http://192.168.254.142:5173. Both addresses depend on the current laptop network. If the laptop's address changes, update the relevant configuration and rebuild the APK.

Keep the laptop awake, the backend running, and both devices on the intended trusted Wi-Fi. Internet access is also required for hosted database and provider services. Restore the normal frontend server after asset compilation. Preserve the private environment files and use additive migrations rather than resetting existing records.

A later hosted release can replace the local address with a stable HTTPS API and use the selected web host for account links. Release builds require HTTPS and an appropriate protected signing identity. Hosting the backend does not automatically update an APK that still contains the old local address; the native client must be rebuilt for that endpoint.

CHAPTER 8
CONCLUSIONS AND RECOMMENDATIONS

Conclusions

AniTrack provides a shared personal anime workspace through the browser and a functional Capacitor Android application. Its principal functions include a persistent watchlist, viewing progress, ratings, favorites, private collections, catalog discovery, anime details, an identification and recommendation assistant, a calendar, and account maintenance. Backend authorization and mandatory verified email define the application boundary.

The recorded automated, database, provider, and physical-device checks support the reviewed local implementation. The follow-up mobile work improved the header and hero composition, corrected account-form hierarchy, and kept public navigation responsive during an unavailable-server condition. The submitted APK is identifiable through its manifest, source revisions, and checksum.

Recommendations and Remaining Work

Before final submission, complete the personal Gmail verification and password-reset delivery check and capture the genuine desktop evidence showing Capacitor installation and the Android project folder. The package contains installation records and capture instructions, but a generated record is not a substitute for the required screenshot.

For use away from the laptop network, deploy the backend to a reachable HTTPS service, configure the public web origin and email-link origin, and rebuild the Android client with the hosted API address. Repeat the critical account, provider, session, and connectivity checks on that hosted candidate. Production signing and broader device coverage should be addressed as release activities after the local assessment requirements are satisfied.

REFERENCES

Express.js Contributors. (n.d.). *Express*. https://expressjs.com/

Final examination and final project instructions. (2026). [Unpublished course announcement supplied to Group 2].

Group 2. (2026a). *AniTrack frontend* [Source code]. GitHub. https://github.com/IanMark101/ANITRACK-FRONTEND

Group 2. (2026b). *AniTrack backend* [Source code]. GitHub. https://github.com/IanMark101/ANITRACK-BACKEND

Group 2. (2026c). *Mobile layout and connection audit*. https://github.com/IanMark101/ANITRACK-FRONTEND/blob/main/docs/EXECUTION-AUDIT-2026-11-06.md

Ionic. (n.d.). *CapacitorHttp*. Capacitor documentation. https://capacitorjs.com/docs/apis/http

Nodemailer. (n.d.). *SMTP transport*. https://nodemailer.com/smtp

PostgreSQL Global Development Group. (n.d.). *What is PostgreSQL?* https://www.postgresql.org/docs/current/intro-whatis.html

Prisma. (n.d.). *What is Prisma ORM?* https://www.prisma.io/docs/orm/v6/overview/introduction/what-is-prisma

Quasar Framework. (n.d.). *What is Capacitor*. https://quasar.dev/quasar-cli-vite/developing-capacitor-apps/introduction/

Vue.js. (n.d.). *Introduction*. https://vuejs.org/guide/introduction.html

APPENDIX A INSTRUCTOR PROCEDURE AND SUBMISSION COMPLIANCE

Eight Step Capacitor Procedure

The instructor's guide describes checking the environment, using the existing frontend, adding Capacitor once, verifying compatible packages, preparing Android, building an APK, installing it on a phone, and testing the resulting application. The following matrix distinguishes directly checked requirements from the command adaptations used for the local build.

Table 7 Compliance with the instructor procedure

Step	Required outcome	Project evidence and disposition
1	Node npm Quasar JDK Studio and SDK installed	Rechecked: Node 26.13.0, npm 12.2.1, project CLI 3.10.3, JDK 21.0.13.0.2, Android Studio, SDK and build-tools 36.
2	Existing web application runs	Frontend configuration exists and the normal local website and API respond.
3	Capacitor mode added once	Existing src-capacitor configuration declares org.anitrack.app and AniTrack. The mode is not added again.
4	Compatible Capacitor major versions	A fresh npm list confirmed core 8.5.3, CLI 8.6.0, and Android 8.5.3. All use major version 8.
5	Android project prepared and synchronized	src-capacitor/android exists. Quasar debug asset preparation and Gradle were used; the exact IDE-opening command was not repeated.
6	Gradle succeeds and generates APK	Fresh incremental assembleDebug passed in 38 seconds with 123 tasks up-to-date. The correct wrapper is .\gradlew.bat.
7	APK installed on an Android phone	Wireless ADB confirmed the installed Infinix APK has the same SHA256 as the submitted file. USB-specific -d was not used.
8	Startup navigation forms features and API work	Recorded browser, real-backend, and physical-phone checks cover the required functions; real Gmail delivery remains separate.

The local debug workflow is a functional adaptation of the guide. It does not claim that the exact production IDE command or USB installation command was executed. The remaining setup screenshot should show the actual project and installed packages using the provided capture instructions.

A fresh recheck also launched the installed app successfully, returned HTTP 200 through Capacitor native HTTP, and opened the login, recovery, and information-navigation

controls. The installed base APK, build output, and submitted file were compared by checksum. Full command records and the two procedural adaptations are documented in setup/INSTRUCTOR-COMPLIANCE.md.

Assessment Evidence Mapping

The course allocates 50 points to the web application and 50 points to the Android application. Table 8 maps the requested evidence to the accompanying files. It records availability rather than assigning a grade.

Table 8 Required submission evidence

Requirement	Accompanying evidence	Status
Web main page and working modules	web/dashboard.png watchlist.png collections.png and anime-detail.png	Included
Installed and running phone application	android/dashboard-running.png and the physical-device build record	Included
At least two working mobile features	android/watchlist.png and android/collections.png	Included
Capacitor installation and Android folder screenshot	setup/CAPTURE-INSTRUCTIONS.md explains the required capture; installation text records the environment.	Screenshot pending
Optional APK	apk/AniTrack-1.0-local-debug.apk with manifest and checksum	Included
Project and group information	Cover page and SUBMISSION-DETAILS.md	Included

Each member must submit individually through their own Google Classroom account during November 16-20, 2026. Members may use the same project screenshots and links. Include the title, section, group number, leader, and all four member names. Check attachment readability and link access before turning in. The instructions do not require a live presentation or online deployment.

APPENDIX B BUILD PROVENANCE AND EVIDENCE NOTES

Reviewed Source and Package

Table 9 Source and build identification

Item	Recorded value
Frontend source revision	20e8eacb3e505cc0ea3d0f31a865fd8f071caecb
Backend source revision	9f4be82f8d27e4ec1b0403a42c02d479e4342aac
Android application	org.anitrack.app; AniTrack; local debug build
Packaged API address	http://192.168.254.142:3001/api
Native toolchain	JDK 21; Gradle 8.14.3; compile and target SDK 36; minimum SDK 24
Capacitor packages	core 8.5.3; Android 8.5.3; CLI 8.6.0; App 8.1.2
APK filename	AniTrack-1.0-local-debug.apk
APK size	10,827,740 bytes
SHA256	c698faa3effbb2f9915123f65d7d1f4d 0906774121e70f074731b10aafe6bb4f

The build manifest and SHA256SUMS.txt accompany the APK. The source revisions identify the tested application code. Documentation revisions may occur afterward without changing that executable.

Evidence Integrity and Remaining Verification

Web images are browser captures. Android images are physical-phone captures, with the current landing, login, and recovery images recorded on November 6. The browser image web/mobile-landing.jpg is a responsive preview and is not used as proof of phone installation. Original-resolution files remain available alongside this report.

Feature evidence used disposable test data. The corresponding test accounts and owned database schemas were removed after the relevant journeys. The retained screenshots document the observed behavior and do not imply that the demonstration data remains in the normal user's account.

Automated token flows and SMTP authentication are recorded as completed; actual inbox receipt remains pending. The Windows capture helper timed out, so the required genuine setup screenshot remains a manual task. The installation record is supplied as text.